COURSE:	CMPS 344 - Programming Languages Department of Computing Sciences, University of Scranton
DATE:	Spring 2022 *(January 26, 2022 - May 20, 2022)*
INSTRUCTOR:	P. M. Jackowitz
OFFICE:	LSC 192
OFFICE HOURS:	As posted (online and office door), and by appointment. To meet remotely you must "Log In" to the Course Web Site (CWS) to access the Zoom links.
TELEPHONE:	(570) 941-6107
EMAIL:	paul.jackowitz@scranton.edu
WWW:	http://www.cs.scranton.edu/~jackowitz

Course Description: (Prerequisite: CMPS 240 *[CMPS 250 Recommended]*) A study of programming languages from both the theoretical and practical perspectives. A survey of major and developing paradigms and languages is undertaken which includes use of specific languages to broaden the student's experience. Implementation is studied through an introduction to language translation along with a study of run-time models and interfaces with virtual machines. (Undergraduate Catalog 2021-2022)

STUDENT LEARNING OUTCOMES: Upon completion of the course, a successful student will have the ability to do each of the following:

1. Knowledge and understanding of the evolutionary history of programming languages and of computing in general.
2. Knowledge of and experience with formal languages, including regular and context-free grammars; specifically as they pertain to the lexical and syntactic aspects of programming languages.
3. Knowledge of and experience with Attribute Grammars as a means of representing semantic elements of programming languages.
4. An in depth understanding of the process of language translation.
5. An appreciation of the concept of "binding" and its use in understanding many aspects of programming and programming languages.
6. Facility with the terminology of programming languages regarding matters such as abstraction, data types, expressions, statements, control flow and subprograms.
7. Exposure to selected relevant constructs present in a variety of programming languages, as a means of appreciating alternatives provided in different programming languages. Languages such as Ada, Eifel, Modula-2, etc. are typical.
8. Fluency in LISP programming as an example of the functional paradigm.
9. Exposure to the Prolog programming language as a means of appreciating the declarative and logic paradigms.

TEXTS: Concepts of Programming Languages - 12th Edition, Robert W. Sebesta, Pearson Higher Education, 2019.

Course Web Site (CWS): http://www.cs.scranton.edu/~jackowit/public/Spring2022/c344
(This site serves as our primary electronic communication tool for this course. You will use it to access required and optional course material and to submit and review assignments. Initially, you must register to obtain full access to the functionality of this site.)

	Worth
GRADING:	
Tests: *(approximate date)*	20%
Week of March 8th	30-40%
Final Exam: *(comprehensive, as per Final Exam Schedule)*	0%-10%
Quizzes: *(as announced)*	
Assignments:	30%
Programming, homework, etc.	10%
Project/Report/Presentation:	

Attendance and Class Participation considered.
(Your attendance at all classes is expected. The accumulation of more than four absences may result in a diminished final course grade.)

PROCEDURES:

Lectures:

- please sit in the same seat for every class meeting
- feel free to ask and answer questions, and to contribute to discussions
- classroom use of electronic devices/gadgets (including computers) is at the full discretion of the instructor. *(Distracting others or yourself will not be tolerated.)*

Tests and Quizzes:

- always announced in advance
- **no make-ups will be given**
- notice must be given if a test must be missed due to serious illness or emergency

Assignments:

- each student is required to do his/her own work on each assignment
- discussions and mutually beneficial collaboration among students is encouraged, but must not be to the point of representing someone else's effort and understanding as your own as this would be considered to be **academic dishonesty** (see Academic Code of Honesty in the Student Handbook).
- academic dishonesty will be dealt with **severely**
- each assignment will have a specified **due date**, and a separate **deadline**
- normally the deadline is later than (typically, but not always two days) the due date
- work submitted **after the due date is considered to be "late"**, will be accepted for grading but may be assessed a penalty (depending upon how late it is, and whether or not worthwhile work had been submitted by the due date).
- work may not be submitted after the **deadline;** it is considered to be **"too late"**, may not be accepted for grading, and may receive a grade of zero (depending upon whether or not worthwhile preliminary work had been submitted prior to the due date or prior to the deadline).
- incomplete work generally will receive a grade much higher than zero
- work not submitted will receive a grade of zero

Other:

- Please read and understand the section, My Reporting Obligations as a Required Reporter, at this link.

List of Topics:

1. Early history and evolution of major programming languages.
2. Languages, Grammars and Computational Models
3. Finite State Machines and Lexical Analysis
4. Push-down Machines, Recursive Descent Parsing and Syntax Analysis
5. Attribute Grammars and Augmented Machines
6. Bindings and Binding Times
7. Names and Symbol Tables
8. Static and Dynamic Scope and Lifetime
9. Data Types - Scalar and Structures
10. Expressions and evaluation
11. Statement Constructs
12. Subprograms and their implementation
13. Functional Programming and a "taste of LISP"
14. Logic Programming and a "taste of Prolog"